

Abstraction and Knowledge in Reinforcement Learning: Evaluating Concept Grounding on Craftax

Max Adamski

Poznań University of Technology, Poland

MAX.ADAMSKI@CS.PUT.POZNAN.PL

Editors: Alessandra Mileo, Andrea Passerini and Cogan Shimizu

Abstract

Symbolic reinforcement learning environments like Craftax provide interpretable observations that are ideal for reasoning. In such gridworlds, tile vectors can be grounded into high-level concepts via an embedding bottleneck to abstract the state space, while encoding task-specific knowledge into them. We investigate concept grounding with multi-label classification, using human-provided binary propositions as labels and a weighted binary cross-entropy loss to handle imbalance in the tile distribution. We integrate concept grounding with Proximal Policy Optimization (PPO) via an additional loss term to allow both the classification loss and the PPO loss to influence embeddings. We evaluate Craftax agents by comparing total episodic rewards (scores), achievement completion rates, and the total number of parameters. Embedding agents show average scores comparable to the baseline, but higher maximum scores, a 2x reduction in total parameters, and better generalization, completing more achievements in a rare level. Grounded agents are more robust to initial conditions, showing that prior knowledge can stabilize reinforcement learning.

1. Introduction

As neuro-symbolic architectures promise improvements in reinforcement learning ([Acharya et al., 2024](#)), we evaluate a simple embedding and concept grounding method on the challenging Craftax benchmark ([Matthews et al., 2024](#)) to gain insight into the role of knowledge in learning. One source of difficulty in Craftax is its wide variety of grid tiles, which are discovered progressively as gameplay ability improves. We hypothesize that learning from abstracted inputs increases generalization (H1) and that prior knowledge guides learning (H2). To test our hypotheses, we force agents to generalize across tiles using a compression bottleneck, and we combine the PPO loss ([Schulman et al., 2017](#)) with an auxiliary multi-label classification loss to ground tile embeddings in an expert-defined taxonomy.

More expressive Logic Tensor Networks ([Serafini and d’Avila Garcez, 2016](#)) have been used to inject prior knowledge and improve shape recognition through symbolic abstraction, although in simpler gridworlds ([Badreddine and Spranger, 2019](#); [Amador and Gierasimczuk, 2025](#)). Concept Bottleneck Models (CBMs) represent an alternative approach to integrating human-defined concepts with neural networks ([Koh et al., 2020](#)), having been used to guide reinforcement learning in Atari games to avoid the misalignment problem ([Delfosse et al., 2024](#)). In general, symbolic interfaces are considered a key enabler of bidirectional human-AI interaction ([Kambhampati et al., 2022](#)), for instance, allowing humans to advise explainable reinforcement learning agents in an interactive setting ([Guan et al., 2021](#)). While expressive and interactive methods are promising, we focus on evaluating the impact of knowledge on reinforcement learning in a complex environment, using a simple method similar to CBMs.

2. Methods

Consider the encoder-decoder architecture shown in Figure 1. It is designed for symbolic gridworld observations consisting of a sequence of tiles $e_1, e_2, \dots, e_N$, where each tile is an interpretable C -dimensional vector (x), and each dimension is either binary or fuzzy ($0 \leq x_i \leq 1$). The encoder is a neural network mapping tile vectors to E -dimensional tile embeddings (z), and the decoder network maps embeddings to D -dimensional logits, which become concept probabilities ($\hat{y}$) after applying the sigmoid function. The ground truth (y) is computed from original tile features (x), using expert-provided binary or fuzzy logic propositions. The embeddings are optimized to predict concepts, and they introduce a compression bottleneck when $E < C$. After independently preprocessing each tile, embeddings $e'_1, e'_2, \dots, e'_N$ are used as the input of the downstream networks.

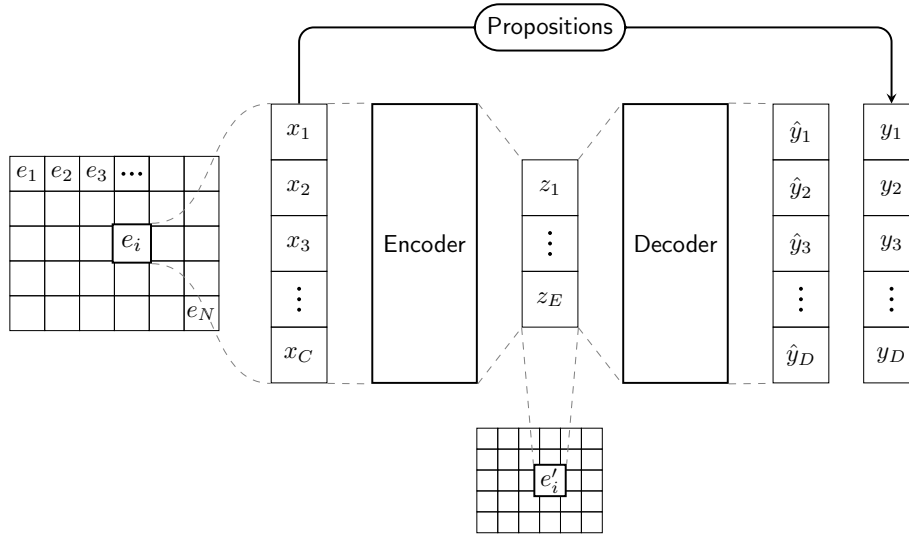


Figure 1: Concept grounding architecture for gridworlds

In Equation (1), we define the loss that the encoder and decoder must minimize so that embeddings predict concepts. It is a standard binary cross-entropy loss for imbalanced multi-label classification, averaged across labels. The concept loss is scaled by the hyperparameter λ_K and is added to the PPO loss ($\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_K \mathcal{L}_K$).

$$\mathcal{L}_K = \frac{1}{D} \sum_{i=1}^D [-w_i y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)] \quad (1)$$

In Equation (2), we define the weighting scheme. Assuming imbalance in concept value distributions, each example is weighted inversely to the mean concept value in the minibatch ($\bar{y}_i$). For binary concepts, the weight is proportional to the number of negative examples in the minibatch (n_i^-), and inversely proportional to the number of positive examples (n_i^+). Negative examples are not weighted, so division by zero is avoided.

$$w_i = \frac{1}{\bar{y}_i} - 1 = \frac{n_i^-}{n_i^+} \quad (2)$$

In Figure 2, we propose a taxonomy of Craftax tiles that contains our knowledge of the game. Craftax observations mainly consist of 83-dimensional tiles $e_1, e_2, \dots, e_{99}$, which are interpreted as an 11×9 grid. For convenience, we assign lowercase names to dimensions $x_1, \dots, x_{83}$, and capitalized names to concepts $y_1, \dots, y_{19}$. Among others, we define the *Ranged* concept, grouping tiles occupied by similar enemies, with the binary proposition $Ranged \iff skeleton \vee orc\ mage \vee gnome\ archer$. We also define concepts in terms of other concepts, like $Enemies \iff Ranged \vee Melee$, and using inequalities, like $Dark \iff light\ level < 0.05$. We implement these propositions as efficient tensor operations.

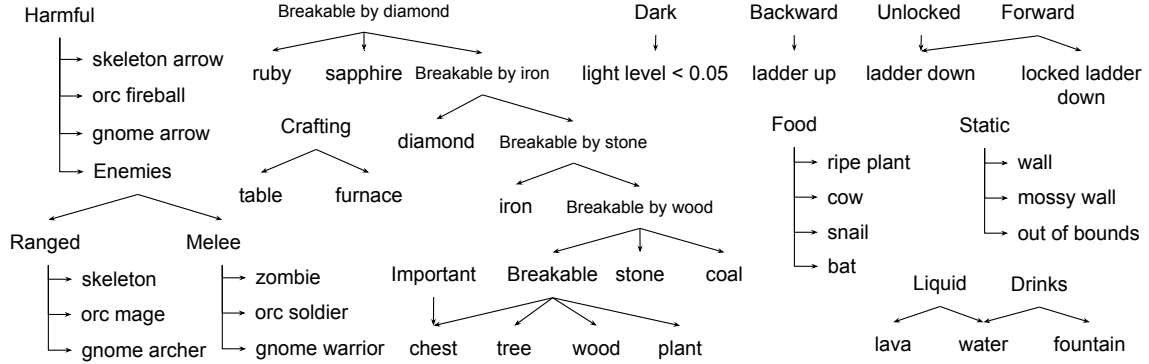


Figure 2: Proposed taxonomy for Craftax

We compare the concept grounding architecture against a baseline non-recursive PPO method. To test H1, we separate the effect of abstraction introduced by the compression bottleneck, and to test H2 we remove guidance with prior knowledge by replacing supervised concept classification with self-supervised tile reconstruction. In total, we evaluate the following four methods:

1. **Baseline:** the agent learns from raw observations.
2. **Compression:** the encoder compresses tiles, but the decoder is removed.
3. **Autoencoder:** compression, but a symmetric decoder reconstructs tiles ($y_i = x_i$).
4. **Grounding:** compression, but a single layer decoder grounds tiles in our taxonomy.

For each method, we train 20 agents for 500 million steps, starting from different random seeds. Notably, we use the same concept loss scale ($\lambda_{\mathcal{K}} = 0.5$) for both the autoencoder and grounding methods. We then evaluate each agent for 100 thousand episodes across 4096 different worlds. During evaluation, agents simply sample actions from their learned policy, which does not require the decoder. After an episode ends, we record the score, defined as the normalized total episodic reward, and the binary vector of completed achievements. For each agent, we divide the number of evaluation episodes where a given achievement is completed, by the total number of evaluation episodes, to get the achievement rate.

3. Results

Table 1 shows the tile dimension, the total agent parameter count (in millions), and the average and maximum score of each method (mean and standard deviation across agents). Embeddings do not change the average score, but they do increase the maximum score. The twofold reduction in agent parameter count is explained in Appendix A.

Table 1: Evaluation summary

Method	Tile Dim.	Params	Avg. Score	Max. Score
Baseline	83	9.54M	11.62% $\pm$ 0.13%	22.75% $\pm$ 0.41%
Compression	32	4.39M	11.82% $\pm$ 0.19%	25.24% $\pm$ 1.00%
Autoencoder	32	4.42M	11.77% $\pm$ 0.17%	25.27% $\pm$ 1.20%
Grounding	32	4.39M	11.87% $\pm$ 0.18%	26.42% $\pm$ 1.15%

In Figure 3, we show the completion rates for analogous achievements in common and rare scenarios (mean across agents and individual values). Embeddings slightly increase achievement rates after entering a common level, and greatly increase analogous achievement rates after entering a rare level.¹ All grounded agents learn to complete analogous achievements at similar rates in both levels, while a few compression and many autoencoder agents show degraded performance in the rare level.

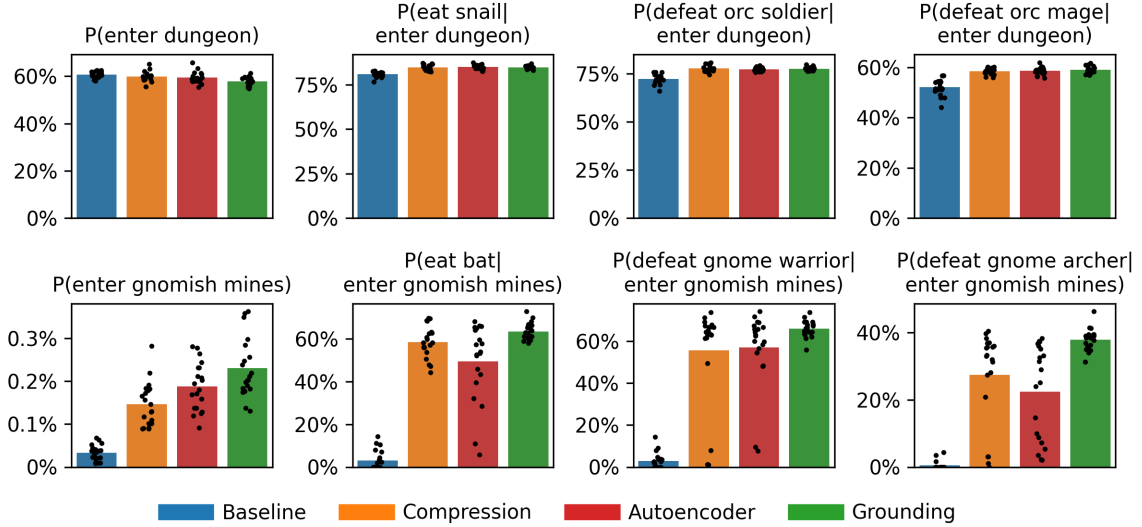


Figure 3: Completion rates for analogous achievements

In Figure 4, we show that compression degrades performance for most agents on two common achievements and one rare achievement, but grounding limits the effect.

1. To compare rates across levels, we show rescaled achievement rates $P(A|B) = P(A)/P(B)$, where completing achievement A requires first completing achievement B .

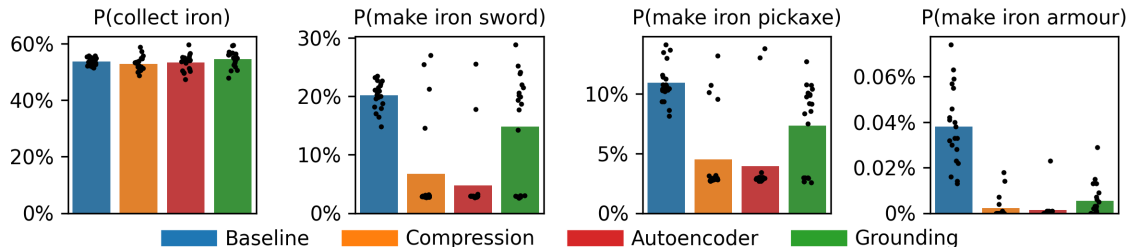


Figure 4: Completion rates for iron-related achievements

4. Conclusions

The average score on Craftax remains at the level of the baseline for all of the investigated embedding methods. However, compressing tiles in Craftax greatly decreased parameter counts and allowed agents to generalize across tiles to complete analogous achievements regardless of level and tile rarity. Additionally, concept grounding stabilized learning with prior knowledge of the symbolic environment.

The difference between compression and concept grounding shows that jointly learning multi-label classification for an expert-designed taxonomy can have a stabilizing effect and can even partially recover performance lost in bottlenecks. Conversely, the autoencoder method shows that learning to reconstruct tiles does not stabilize learning in a symbolic environment where tiles do not contain visual patterns.

We see that learning from abstracted inputs can increase generalization (H1), as supported by significant improvements in completion rates for a class of Craftax achievements and dramatic improvements in a rare level where new tiles are introduced. We also find that prior knowledge can guide learning (H2), as shown in a rare level where unguided agents can fail to complete analogous achievements simply because of a random seed, as well as in a common level where most unguided embedding agents fail to complete iron-related achievements, while many grounding agents succeed.

Reinforcement learning with PPO can be unstable, as shown by the high variance of average scores across training runs on some Atari games (Schulman et al., 2017), and as we see with embedding agents on some Craftax achievements. In this context, we consider concept grounding to be a semantic regularizer, as it reduces sensitivity to random seeds. However, it has limited applicability by assuming a symbolic environment and requiring additional modeling and knowledge engineering.

Future work may investigate more expressive architectures and knowledge bases with recurrent Craftax models; though they achieve higher scores, they remain insufficient to fully solve this benchmark, even on a large 10 billion step budget.

Acknowledgments

We thank Jędrzej Potoniec and Wojciech Kotłowski for their support, and the Poznań University of Technology for GPU access.

References

- Kamal Acharya, Waleed Raza, Carlos M. J. M. Dourado Júnior, Alvaro Velasquez, and Houbing Herbert Song. Neurosymbolic reinforcement learning and planning: A survey. *IEEE Transactions on Artificial Intelligence*, 5(5):1939–1953, 2024.
- Ivo Amador and Nina Gierasimczuk. Symdqn: Symbolic knowledge and reasoning in neural network-based reinforcement learning. In *NeSy*, Proceedings of Machine Learning Research, pages 70–85. PMLR, 2025.
- Samy Badreddine and Michael Spranger. Injecting prior knowledge for transfer learning into reinforcement learning algorithms using logic tensor networks. In *NeSy@IJCAI*, 2019.
- Quentin Delfosse, Sebastian Sztiwrtnia, Mark Rothermel, Wolfgang Stammer, and Kristian Kersting. Interpretable concept bottlenecks to align reinforcement learning agents. In *NeurIPS*, 2024.
- Lin Guan, Mudit Verma, Sihang Guo, Ruohan Zhang, and Subbarao Kambhampati. Widening the pipeline in human-guided reinforcement learning with explanation and context-aware data augmentation. In *NeurIPS*, pages 21885–21897, 2021.
- Subbarao Kambhampati, Sarath Sreedharan, Mudit Verma, Yantian Zha, and Lin Guan. Symbols as a lingua franca for bridging human-ai chasm for explainable and advisable AI systems. In *AAAI*, pages 12262–12267. AAAI Press, 2022.
- Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *ICML*, Proceedings of Machine Learning Research, pages 5338–5348. PMLR, 2020.
- Michael T. Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Thomas Jackson, Samuel Coward, and Jakob Nicolaus Foerster. Craftax: A lightning-fast benchmark for open-ended reinforcement learning. In *ICML*, Proceedings of Machine Learning Research, pages 35104–35137. PMLR, 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Luciano Serafini and Artur S. d’Avila Garcez. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. In *NeSy@HLAI*, CEUR Workshop Proceedings. CEUR-WS.org, 2016.

Appendix A. Architectural details

Our downstream actor and critic networks have 3 layers, each with 512 neurons and ReLU activation. Craftax has an 8268-dimensional observation space ($9 \times 11 \times 83$ for tiles and 51 for global state), so the actor and critic contain $8268 \times 512 \times 2$ weights in their first layers, which accounts for the majority of agent parameters. The encoder accepts 83-dimensional inputs, consists of 3 layers, each with 83 neurons and ReLU activation, and outputs 32-dimensional

embeddings. Embedding the tiles results in a 3219-dimensional observation space, which proportionately reduces the number of weights in the first layers of the actor and critic.

Appendix B. Training details

We train each agent for 500 million steps across 1024 different environments. We perform rollouts for 64 steps, resulting in batches of 65536 steps.² Each batch is divided into 16 minibatches of 4096 steps. Notably, each step contains an observation with 99 tiles, meaning that the encoder is trained on large minibatches of tiles. To prevent exploding gradients, we apply gradient clipping with a maximum norm of 1.

In Figure 5, we show achievement rates during training (mean and standard deviation across agents). To reduce noise, we apply a running average with a window size of 40 batches, resulting in 190 data points per plot.

We optimize the agent end-to-end with AdamW, with a constant learning rate throughout training ($\eta = 2 \cdot 10^{-4}$), and using default values for the remaining optimizer hyperparameters.³ We reuse each batch for 3 epochs to improve sample efficiency, and we use the same values as Matthews et al. (2024) for generalized advantage estimation ($\lambda = 0.8$), the discount factor ($\gamma = 0.99$), and PPO hyperparameters for policy update clipping ($\epsilon = 0.2$), scaling the value loss ($c_1 = 0.5$), and scaling the entropy bonus ($c_2 = 0.01$).

Appendix C. Evaluation details

We consider a set of 42 achievements. Because no agents advanced beyond the *gnomish mines* level, we exclude achievements that can only be completed in later levels. For completeness, we present the evaluation results for all considered achievements in Figure 6.

For each method, we train a population of agents ($n = 20$) using different random seeds. We then apply the non-parametric, two-sided Mann-Whitney U test to support our claims of statistically significant changes in achievement completion rates relative to the baseline method. A significant result for a given achievement indicates that the two methods have distinct completion rate distributions, accounting for the effects of random initialization during training and randomness of the environment.

We perform three families of tests, independently comparing the baseline method’s achievement rates with those of the compression, autoencoder, and grounding methods. For each family, we apply the Bonferroni-Holm correction to control the family-wise error rate and reduce the probability of false-positive findings when performing multiple comparisons.

In Table 2, we report the mean and standard deviation of the achievement rates (in %) across the baseline agents. For the embedding methods, we report the change in the mean achievement rate in percentage points (Δ p.p.). For each achievement, we underline statistically significant changes ($p < 0.05$) and highlight the change that results in the highest mean completion rate, indicating the best embedding method for that metric. For certain achievements, a negative change is highlighted, which indicates that the baseline outperforms the embedding methods. To reduce noise, we omit highlighting for achievements with no statistically significant changes.

2. We prevent illegal actions during rollouts by setting their logits to $-\infty$ before applying softmax.

3. See <https://docs.pytorch.org/docs/2.9/generated/torch.optim.AdamW.html>

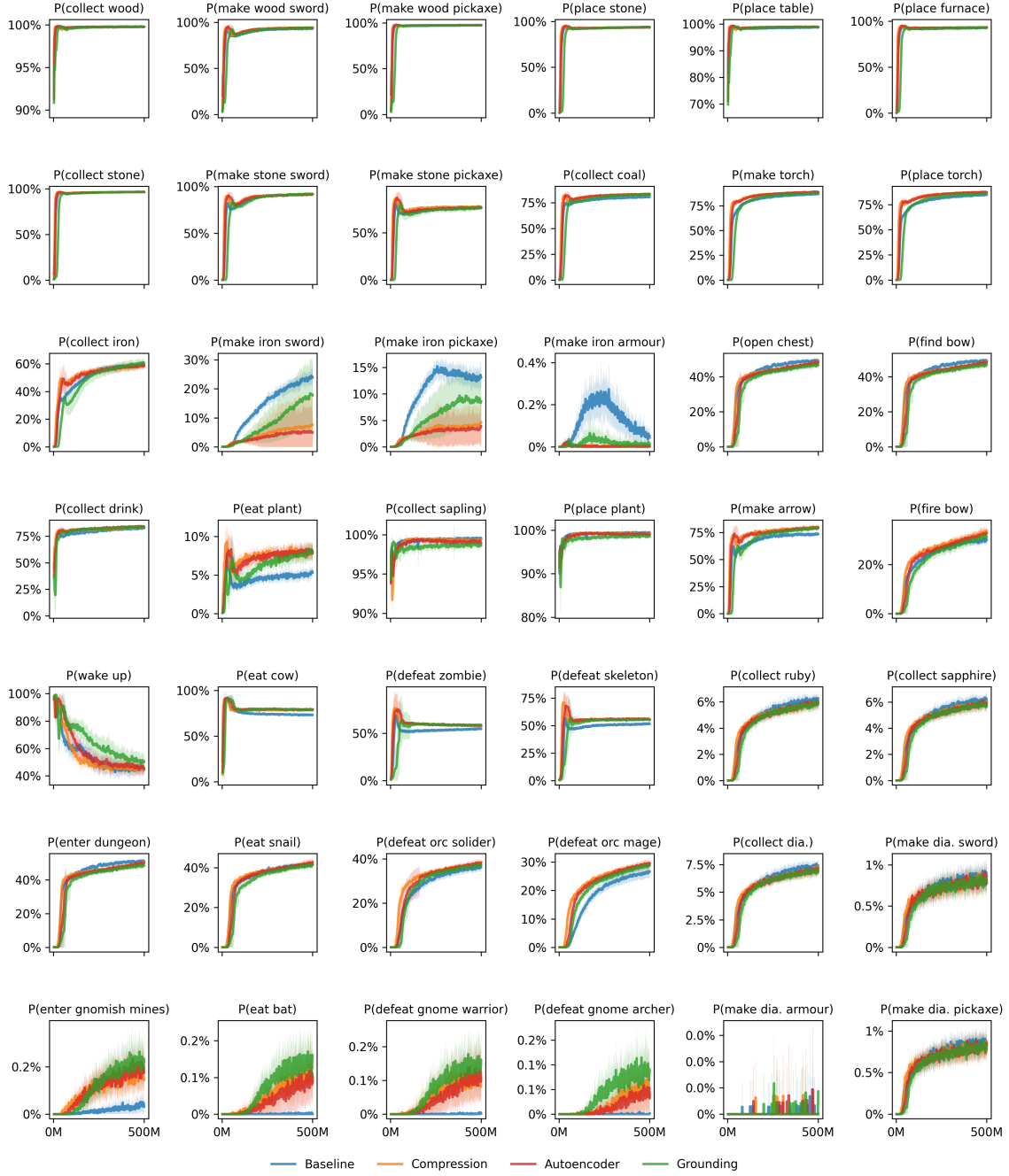


Figure 5: Completion rates for all considered achievements during training

EVALUATING CONCEPT GROUNDING ON CRAFTAX

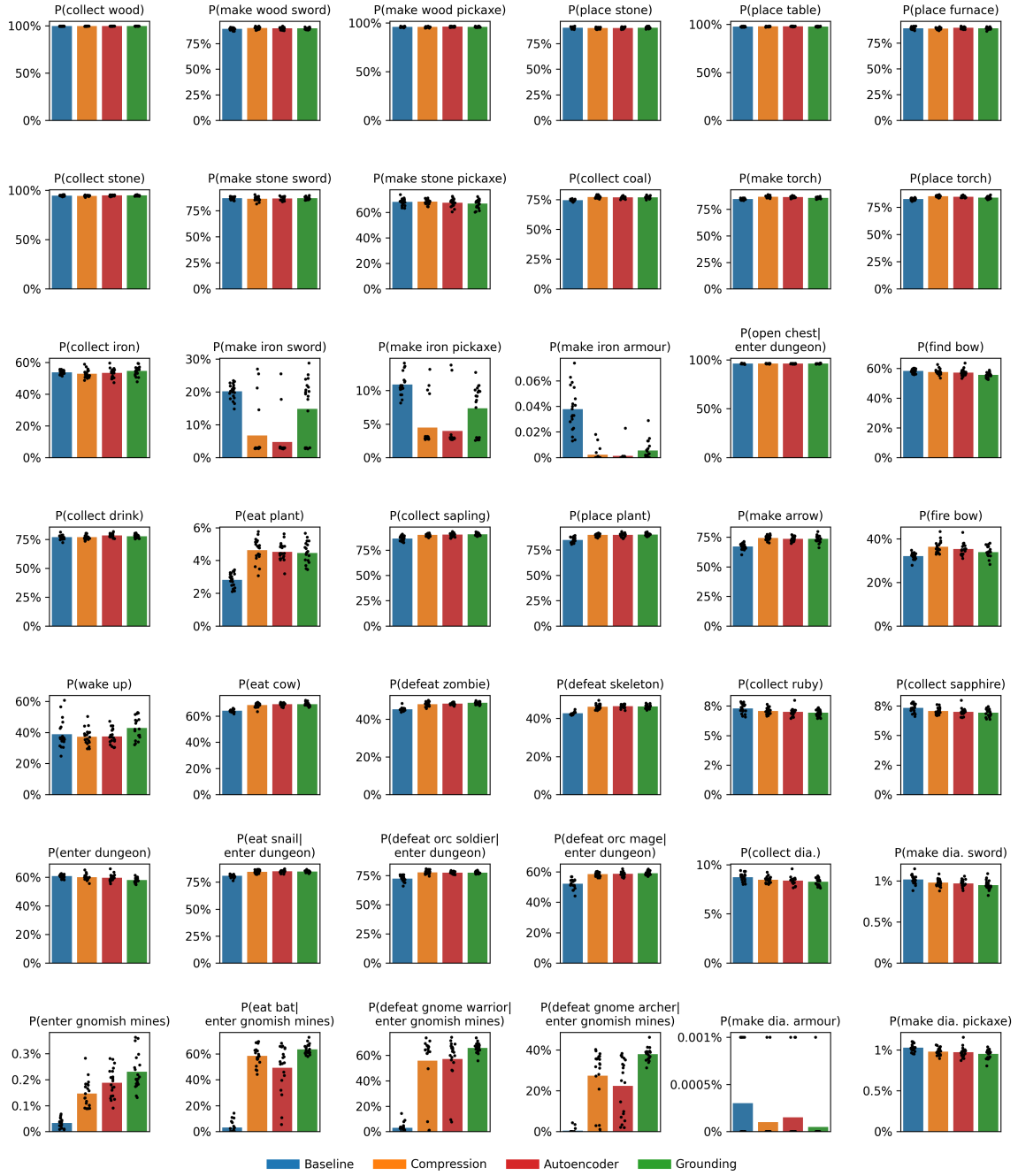


Figure 6: Evaluation results for all considered achievements

Table 2: Statistical tests

Metric	Baseline	Compression		Autoencoder		Grounding	
	$\mu \pm \sigma$ (%)	Δ p.p.	p-val	Δ p.p.	p-val	Δ p.p.	p-val
P(collect wood)	99.53 $\pm$ 0.11	0.06	0.9781	0.09	0.3332	0.05	1.0000
P(make wood sword)	89.19 $\pm$ 1.05	1.00	0.1311	0.58	1.0000	0.58	1.0000
P(make wood pickaxe)	95.80 $\pm$ 0.46	-0.01	1.0000	0.28	0.9444	0.20	1.0000
P(place stone)	90.51 $\pm$ 0.80	-0.39	1.0000	-0.25	1.0000	0.05	1.0000
P(place table)	97.65 $\pm$ 0.40	0.26	0.5043	0.33	0.1420	0.14	1.0000
P(place furnace)	89.46 $\pm$ 1.21	-0.30	1.0000	0.40	1.0000	-0.01	1.0000
P(collect stone)	94.61 $\pm$ 0.54	-0.27	1.0000	0.10	1.0000	0.10	1.0000
P(make stone sword)	86.87 $\pm$ 1.32	-0.34	1.0000	-0.24	1.0000	0.21	1.0000
P(make stone pickaxe)	68.28 $\pm$ 2.62	0.15	1.0000	-0.66	1.0000	-1.31	1.0000
P(collect coal)	74.61 $\pm$ 0.79	2.59	0.0001	<u>2.29</u>	0.0001	<u>2.38</u>	0.0001
P(make torch)	84.78 $\pm$ 0.77	1.98	0.0001	<u>1.87</u>	0.0001	<u>1.09</u>	0.0091
P(place torch)	82.39 $\pm$ 0.98	2.50	0.0001	<u>2.21</u>	0.0001	<u>1.40</u>	0.0112
P(collect iron)	53.64 $\pm$ 1.31	-0.89	1.0000	-0.40	1.0000	0.94	1.0000
P(make iron sword)	20.16 $\pm$ 2.31	<u>-13.42</u>	0.0016	<u>-15.41</u>	0.0001	-5.35	1.0000
P(make iron pickaxe)	10.93 $\pm$ 1.57	<u>-6.44</u>	0.0002	<u>-6.98</u>	0.0002	-3.58	0.0086
P(make iron armour)	0.04 $\pm$ 0.02	<u>-0.04</u>	0.0001	<u>-0.04</u>	0.0001	-0.03	0.0001
P(find bow)	58.26 $\pm$ 1.38	-0.70	1.0000	-1.13	0.3332	<u>-2.61</u>	0.0008
P(make arrow)	67.17 $\pm$ 2.59	7.21	0.0001	<u>6.45</u>	0.0001	<u>6.36</u>	0.0001
P(fire bow)	32.12 $\pm$ 1.50	4.24	0.0001	<u>3.28</u>	0.0010	<u>1.89</u>	0.0364
P(collect drink)	76.88 $\pm$ 1.80	0.00	1.0000	1.37	0.1829	0.78	1.0000
P(collect sapling)	86.26 $\pm$ 2.56	<u>3.74</u>	0.0003	<u>3.94</u>	0.0005	4.22	0.0001
P(place plant)	84.85 $\pm$ 2.62	<u>4.95</u>	0.0001	<u>5.15</u>	0.0001	5.26	0.0001
P(eat plant)	02.81 $\pm$ 0.44	1.81	0.0001	<u>1.71</u>	0.0001	<u>1.65</u>	0.0001
P(wake up)	38.81 $\pm$ 8.64	-1.61	1.0000	-1.35	1.0000	4.04	0.7912
P(eat cow)	64.08 $\pm$ 1.03	<u>4.33</u>	0.0001	<u>4.98</u>	0.0001	5.05	0.0001
P(defeat zombie)	45.29 $\pm$ 0.90	<u>2.78</u>	0.0001	<u>2.99</u>	0.0001	3.41	0.0001
P(defeat skeleton)	42.70 $\pm$ 0.77	<u>3.47</u>	0.0001	3.72	0.0001	<u>3.50</u>	0.0001
P(enter dungeon)	60.60 $\pm$ 1.42	-0.79	1.0000	-1.17	0.2903	<u>-2.74</u>	0.0004
P(open chest dungeon)	96.14 $\pm$ 0.28	0.09	1.0000	-0.01	1.0000	0.04	1.0000
P(eat snail dungeon)	80.77 $\pm$ 1.45	<u>3.78</u>	0.0001	4.24	0.0001	<u>4.04</u>	0.0001
P(defeat orc soldier ...)	72.20 $\pm$ 2.39	5.40	0.0001	<u>5.07</u>	0.0001	<u>5.23</u>	0.0001
P(defeat orc mage ...)	52.24 $\pm$ 3.23	<u>6.19</u>	0.0001	<u>6.36</u>	0.0001	6.82	0.0001
P(enter g. mines)	0.03 $\pm$ 0.02	<u>0.11</u>	0.0001	<u>0.16</u>	0.0001	0.20	0.0001
P(eat bat mines)	3.14 $\pm$ 4.43	<u>55.35</u>	0.0001	<u>46.19</u>	0.0001	60.26	0.0001
P(defeat g. warrior ...)	2.96 $\pm$ 3.66	<u>52.78</u>	0.0001	<u>54.11</u>	0.0001	62.90	0.0001
P(defeat g. archer ...)	0.48 $\pm$ 1.22	<u>26.93</u>	0.0001	<u>22.00</u>	0.0001	37.44	0.0001
P(collect ruby)	7.30 $\pm$ 0.38	-0.23	0.9584	-0.30	0.1852	-0.37	0.0635
P(collect sapphire)	7.33 $\pm$ 0.35	-0.26	0.3000	-0.33	0.0512	<u>-0.41</u>	0.0175
P(collect diamond)	8.72 $\pm$ 0.39	-0.27	0.5005	-0.35	0.0846	<u>-0.47</u>	0.0183
P(make dia. sword)	1.02 $\pm$ 0.05	-0.04	0.2832	-0.05	0.0581	<u>-0.07</u>	0.0112
P(make dia. pickaxe)	1.03 $\pm$ 0.04	-0.05	0.0867	<u>-0.06</u>	0.0213	<u>-0.08</u>	0.0021
P(make dia. armour)	3e-4 $\pm$ 5e-4	-2.0e-4	1.0000	-1.5e-4	1.0000	-2.5e-4	0.6294